

1. Generación de Elementos

Como parámetros se reciben el orden m y el polinomio primitivo

$$P(X) = a_0 + a_1X + \cdots + a_mX^m,$$

representado mediante el número binario $a_0 a_1 \dots a_{m-1}$, donde el coeficiente $a_m = 1$ se encuentra implícito y $a_i \in GF(2)$.

1.1. Multiplicación por α

Para generar los elementos del campo, basta con comenzar por el elemento 1 y multiplicar sucesivamente por α . Sin embargo, para obtener la representación de cada elemento como una tupla, es necesario utilizar la relación $P(\alpha) = 0$.

La multiplicación por α implica desplazar hacia la derecha los bits de la tupla. Por ejemplo, $\alpha = 10 \dots 0$ y $\alpha^2 = 01 \dots 0$. No obstante, cuando se tiene $\alpha^{m-1} = 00 \dots 1$, al multiplicar por α se obtiene α^m . En este caso, no basta con realizar el desplazamiento, ya que el resultado sería $00 \dots 0$. Por lo tanto, se debe descartar el bit 1 correspondiente a α^{m-1} e incorporar el término α^m .

Para representar α^m , se utiliza nuevamente la condición $P(\alpha) = 0$:

$$P(\alpha) = a_0 + a_1\alpha + \cdots + \alpha^m = 0 \implies \alpha^m = a_0 + a_1\alpha + \cdots + a_{m-1}\alpha^{m-1}.$$

Por lo tanto, en forma de tupla, α^m equivale a la tupla recibida como parámetro para representar el polinomio primitivo.

```

1 def _multiply_by_alpha(self, value):
2
3     overflow = value & 1
4
5     value >>= 1
6
7     if overflow:
8
9         value ^= self.primitive_polynomial
10
11     return value

```

Esta función implementa el procedimiento descrito. La variable `overflow` detecta si el valor original posee un bit en la última posición, lo que indica la presencia del término α^{m-1} . Luego, los bits se desplazan hacia la derecha y, en caso de detectarse `overflow`, el resultado se suma con el polinomio recibido mediante una operación XOR bit a bit.

1.2. Construcción de las Tablas de Logaritmos y Antilogaritmos

Para construir las tablas, se comienza con el elemento 1, cuyo exponente asociado es 0. A continuación, se itera sobre todos los exponentes posibles, desde 0 hasta $2^m - 2$.

En cada iteración, se almacena en la tabla de antilogaritmos, en la posición correspondiente al exponente, el valor de la tupla asociada. De manera análoga, en la tabla de logaritmos se almacena, en la posición correspondiente al valor de la tupla, el exponente asociado. El siguiente elemento se obtiene multiplicando el valor actual por α .

Durante la construcción deben realizarse dos verificaciones. En primer lugar, si un elemento aparece más de una vez antes de completar la iteración, el polinomio utilizado no es primitivo. En segundo lugar, una vez recorridos todos los exponentes, el elemento obtenido debe volver a ser 1, de acuerdo con la relación

$$\alpha^{2^m - 1} = 1.$$

Si esta condición no se cumple, el polinomio tampoco es primitivo.

```

1 def _build_tables(self):
2
3     exponent_table = [0] * (self.order - 1)
4     logarithm_table = [None] * self.order
5
6     current = self.one_value
7
8     for exponent in range(self.order - 1):
9
10        if logarithm_table[current] is not None:
11            raise ValueError(
12                "El polinomio ingresado no es primitivo."
13            )
14
15        exponent_table[exponent] = current
16        logarithm_table[current] = exponent
17
18        current = self._multiply_by_alpha(current)
19
20    if current != self.one_value:
21        raise ValueError(
22            "El polinomio ingresado no es primitivo."
23        )
24
25    return exponent_table, logarithm_table

```

1.3. Constructor

El constructor de la clase genera, en primer lugar, las tablas de logaritmos y antilogaritmos a partir del polinomio recibido. Luego, itera sobre todos los exponentes posibles para construir los elementos del campo.

Finalmente, la lista completa de elementos se almacena en cada uno de ellos. De esta manera, todos los elementos pueden acceder a los demás elementos pertenecientes al mismo campo de Galois y realizar operaciones entre sí.

```

1 class GaloisField:
2
3     def __init__(
4         self,
5         m,
6         primitive_polynomial,
7         value=0,
8         exponent_table=None,
9         logarithm_table=None
10    ):
11        if not isinstance(m, int) or m <= 0:
12            raise ValueError("m debe ser un entero positivo.")
13
14        if not isinstance(primitive_polynomial, int):
15            raise TypeError(
16                "El polinomio primitivo debe representarse como un entero."
17            )
18
19        self.m = m
20        self.primitive_polynomial = primitive_polynomial
21        self.order = 2 ** m
22        self.value = value
23
24        if not 0 <= primitive_polynomial < self.order:
25            raise ValueError(

```

```

26         f"El polinomio primitivo debe estar entre "
27         f"0 y {self.order - 1}."
28     )
29
30     if not isinstance(value, int):
31         raise TypeError(
32             "El valor del elemento debe ser un entero."
33         )
34
35     if not 0 <= value < self.order:
36         raise ValueError(
37             f"El elemento debe estar entre 0 y {self.order - 1}."
38         )
39
40     self.one_value = 1 << (self.m - 1)
41
42     if exponent_table is None or logarithm_table is None:
43         self.exponent_table, self.logarithm_table = (
44             self._build_tables()
45         )
46
47     self.elements = [
48         GaloisField(
49             m=m,
50             primitive_polynomial=primitive_polynomial,
51             value=i,
52             exponent_table=self.exponent_table,
53             logarithm_table=self.logarithm_table
54         )
55         for i in range(self.order)
56     ]
57
58     for element in self.elements:
59         element.elements = self.elements
60
61     else:
62         self.exponent_table = exponent_table
63         self.logarithm_table = logarithm_table

```

2. Operaciones

En primer lugar, se implementó una función que permite verificar que ambos operandos pertenezcan al mismo campo de Galois. Para ello, se comprueba que posean el mismo orden y el mismo polinomio primitivo.

Luego, se sobrecargó el operador `[,]` para utilizarlo como método de acceso a los distintos elementos del campo según el valor de su tupla.

La suma se implementó mediante una operación XOR bit a bit. Este procedimiento también se utiliza para la resta, ya que ambas operaciones son equivalentes en los campos de Galois $GF(2^m)$.

Las operaciones de multiplicación, inversión y división se implementaron sumando, negando y restando, respectivamente, los exponentes en módulo $m - 1$, mediante el uso de las tablas generadas previamente. De manera análoga, la potenciación se realizó escalando el exponente correspondiente.

Finalmente, para comparar dos elementos, se verifica que ambos posean el mismo valor de tupla.

```

1 def _check_other(self, other):
2
3     if not isinstance(other, GaloisField):
4         raise TypeError(
5             "La operaci n requiere otro elemento de Galois."
6         )

```

```

7
8     if (
9         self.m != other.m
10        or self.primitive_polynomial
11        != other.primitive_polynomial
12    ):
13        raise ValueError(
14            "Los elementos pertenecen a campos diferentes."
15        )
16
17 def __getitem__(self, value):
18
19     if not isinstance(value, int):
20         raise TypeError(
21             "El valor del elemento debe ser un entero."
22         )
23
24     if not 0 <= value < self.order:
25         raise ValueError(
26             f"El elemento debe estar entre 0 y {self.order - 1}."
27         )
28
29     return self.elements[value]
30
31 def __add__(self, other):
32
33     self._check_other(other)
34
35     result = self.value ^ other.value
36
37     return self.elements[result]
38
39 def __sub__(self, other):
40
41     return self + other
42
43 def __mul__(self, other):
44
45     self._check_other(other)
46
47     if self.value == 0 or other.value == 0:
48         return self.elements[0]
49
50     exponent_a = self.logarithm_table[self.value]
51     exponent_b = self.logarithm_table[other.value]
52
53     result_exponent = (
54         exponent_a + exponent_b
55     ) % (self.order - 1)
56
57     result_value = self.exponent_table[result_exponent]
58
59     return self.elements[result_value]
60
61 def inverse(self):
62
63     if self.value == 0:
64         raise ZeroDivisionError(
65             "El elemento 0 no tiene inverso multiplicativo."
66         )
67
68     exponent = self.logarithm_table[self.value]
69

```

```

70     inverse_exponent = (
71         -exponent
72     ) % (self.order - 1)
73
74     result_value = self.exponent_table[inverse_exponent]
75
76     return self.elements[result_value]
77
78 def __floordiv__(self, other):
79
80     self._check_other(other)
81
82     if other.value == 0:
83         raise ZeroDivisionError(
84             "No se puede dividir por cero."
85         )
86
87     if self.value == 0:
88         return self.elements[0]
89
90     exponent_a = self.logarithm_table[self.value]
91     exponent_b = self.logarithm_table[other.value]
92
93     result_exponent = (
94         exponent_a - exponent_b
95     ) % (self.order - 1)
96
97     result_value = self.exponent_table[result_exponent]
98
99     return self.elements[result_value]
100
101 def __truediv__(self, other):
102
103     return self // other
104
105 def __pow__(self, exponent):
106
107     if not isinstance(exponent, int):
108         raise TypeError(
109             "El exponente debe ser un entero."
110         )
111
112     if exponent < 0:
113         raise ValueError(
114             "El exponente debe ser mayor o igual que cero."
115         )
116
117     # Por convención,  $a^0 = 1$ .
118     if exponent == 0:
119         return self.elements[self.one_value]
120
121     # Para  $n > 0$ ,  $0^n = 0$ .
122     if self.value == 0:
123         return self.elements[0]
124
125     base_exponent = self.logarithm_table[self.value]
126
127     result_exponent = (
128         base_exponent * exponent
129     ) % (self.order - 1)
130
131     result_value = self.exponent_table[result_exponent]
132

```

```
133     return self.elements[result_value]
134
135 def __eq__(self, other):
136
137     if not isinstance(other, GaloisField):
138         return False
139
140     return (
141         self.m == other.m
142         and self.primitive_polynomial
143         == other.primitive_polynomial
144         and self.value == other.value
145     )
146
147 def __int__(self):
148
149     return self.value
150
151 def __repr__(self):
152
153     binary = format(self.value, f"0{self.m}b")
154
155     return f"GF({self.value}, Ob{binary})"
```